

Cloud Computing Resource Allocation Problems

Student: Ty Riekert
Supervisor: Elham Mardaneh
Co-supervisor: Tony Mathew

February 2026

1 Pre-Research

1.1 Cloud Computing Resource Allocation Problems

Cloud computing refers to the use of computing services and resources delivered over the internet [1]. Unlike relying on traditional local computers and hard drives, entities can buy computing resources like servers, storage, computing power and more, allowing for easy and on-demand network access of computing services [2, 3]. However, resource allocation (RA) has become a critical issue for cloud computing, especially in Infrastructure-as-a-Service (IaaS) [3].

Generally, and most fundamentally, Virtual Machine (VM) placement is a core subproblem of cloud-based RA, which involves mapping VMs to physical hosts based on CPU, memory, and storage availability. Effective VM placement directly impacts utilisation, energy consumption and more — making it a central RA area of research [2].

Task scheduling is another major subproblem within RA. The primary objective of these techniques is to divide workload evenly across all available resources whilst handling key objectives and issues like reducing execution and response time of servers [4].

Autoscaling addresses the need to dynamically adjust resource capacity in response to workload fluctuations. Cloud systems rely heavily on autoscaling to maintain efficiency, critical for aligning resource supply with demand in real time [5]. It can often be considered as predictive modelling and online decision-making approach.

Load Balancing ensures that incoming tasks are distributed evenly across computing nodes to maintain stable performance and prevent hotspots [6]. It is closely linked to VM placement.

1.2 Formal Models for Cloud Resource Allocation Problems

Consider the classical bin packing problem, we are given a list $I = \{i_1, i_2, \dots, i_n\}$ of real numbers in the range $[0, 1)$. The goal is to place them in a minimum number of bins so that no bin holds numbers summing to more than 1 (the capacity of each bin cannot be broken) [7]. This problem is well-known to be NP-hard.

Cloud RA problems are typically formalised using well-studied combinatorial optimisation models. The most widely used to model VM placement are Vector Bin Packing (VBP) and Multidimensional Bin Packing (MDBP) [7]. In a d -dimensional VBP, each item (represented as a set of resource demands like CPU, memory, etc.) is a d -dimensional vector that needs to be packed into unit vector bins (servers), each of some capacity [8]. (We obtain a polynomial-time algorithm with an asymptotic approximation ratio of $\ln(1.5) + 1 \approx 1.405$ for 2-D VBP [7].) Khan [7] proposes that VM placement is structurally equivalent to bin packing and vector bin packing, therefore VBP is a dominant model for VM placement.

Multidimensional Bin Packing (MDBP) has the objective to maximise a subset of workloads to sum total value (e.g., revenue and throughput) under multiple constraints. It works well when not all requests can be accommodated [9].

Cloud workloads arrive and depart unpredictably, making online bin packing a natural model for real-time VM placement [7].

Scheduling and load balancing models focus on assigning tasks to optimise objectives such as makespan (total time required to complete all tasks) and throughput. Surveys by [2] indicate that scheduling and VM placements are tightly coupled, both aiming to distribute workloads efficiently. It is essential for maintaining performance in large-scale cloud systems [1].

Multiobjective Bin Packing is necessary to capture real-world trade-offs, and classical single-objective models are insufficient for modern datacenters [3].

1.3 Limitations of Classical Methods

- Scalability
- Adaptability
- Generalisation
- Multi-objective complexity

1.4 Cloud RA Metrics

There are many different metrics of which RA tries to optimise. One such is **energy-efficiency optimisation**, which aims to reduce the power consumption of data centres [10, 11], with sustainability and environmental impact being a significant focus in the past few years.

[8] summarises some of the most challenging research issues in cloud computing. The first is automated service provisioning (dynamic resource provisioning), which concerns allocating and de-allocating resources from the cloud. The second is virtual machine migration. The third is server consolidation, which aims to maximise resource utilisation while minimising energy consumption in a cloud computing environment without hurting application performance. Improving energy efficiency is another major issue in cloud computing. The fourth is traffic management and analysis, which focuses on optimising customer experiences.

[12] proposes many different metrics for cloud KPIs:

- **Service response time**: measures the efficiency of service availability.
 - Average response time: $\frac{\sum_{i=1}^n T_i}{n}$, where T_i is the time between when user i requests a service and when it becomes available.
 - Maximum response time: the maximum promised response time by the provider.
 - Response time failure rate: $100 \times \frac{n'}{n}$, where n' is the number of occasions when the provider failed to meet the promised maximum response time.
- **Sustainability** (not relevant): defined in terms of environmental impact such as carbon footprint. Some well-known metrics include:
 - **DCiE** and **PUE**: Data Center Infrastructure Efficiency (percentage of total facility power going to IT equipment). PUE is the inverse of DCiE.
 - **DPPE** (Data Center Performance per Energy): correlates datacenter performance with carbon emissions.

$$DPPE = ITEU \times ITEE \times \frac{1}{PUE} \times \frac{1}{1 - GEC}$$

where:

$$ITEU = \frac{\text{Total Measured IT Energy (kWh)}}{\text{Total Specified IT Energy (kWh)}}$$

$$ITEE = \frac{a \sum \text{Server Capacity} + b \sum \text{Storage Capacity} + c \sum \text{Network Capacity}}{\text{Total Specified IT Energy (kWh)}}$$

$$GEC = \frac{\text{Green Energy}}{\text{Total Datacenter Power Consumption}}$$

- **Suitability** (not relevant)
- **Accuracy** (not relevant)

- **Transparency** (not relevant)
- **Throughput and Efficiency**
- **Makespan**: maximum completion time of all tasks
- **Throughput**: number of completed tasks per unit time
- **CPU/Memory Utilisation**: percentage of resource usage
- **SLA Violation Rate**: fraction of requests violating Service Level Agreement (SLA) constraints
- **Latency**: time between request and response
- **Energy Consumption**: total energy used by active servers
- **Resource Waste**: allocated but unused resources
- **Load Distribution Efficiency**: evenness of load across resources

2 Problem Formulations

2.1 RA Formulation

The RA problem can be treated as a two-level vector bin packing problem [2]. The first level concerns the admission of new VM requests and placing VMs on hosts. The second level concerns optimising the current VM allocation.

Objectives:

- **Energy Consumption (EC)**: One of the characteristics that must be addressed (and most commonly studied) is energy consumption.
- **Load balancing (LB)**: Achieving an even distribution of tasks among VMs to ensure optimal resource utilization and avoid overloading or underutilization [2]
- **Minimising Response Time (RT)**: Assigning tasks to VMs and VMs to PMs (physical machines) in a way that reduces overall response time whilst improving the overall performance of the system.
- **Maximising Resource utilization (RU)**: Effectively utilizing available resources

Constraints:

- **Resource Availability**:
- **Security requirements**
- **Network communication**
- **Cost considerations**
- **Hardware variations**

RA Classifications When doing a RA problem, there are many different ways they can be classified [2]. (THE FOLLOWING IS COPY AND PASTED).

- **Static vs. Dynamic:** RA techniques can be classified into static and dynamic allocation. Static allocation involves pre-allocating resources to users and applications based on their expected workload, while dynamic allocation involves allocating resources in real-time based on current demand
- **Centralized vs. Decentralized:** RA techniques can also be classified into centralized and decentralized allocation. In centralized allocation, a central controller makes decisions about RA, while in decentralized allocation, decisions are made by individual nodes or agents
- **Probabilistic vs. Deterministic:** RA techniques can be classified as probabilistic or deterministic. Probabilistic techniques use statistical methods to make decisions, while deterministic techniques use a fixed set of rules.
- **Load Balancing:** Load balancing techniques aim to distribute the workload evenly across multiple servers or VMs, which can improve resource utilization and reduce response time
- **Scheduling:** Scheduling techniques aim to schedule tasks and jobs to optimize the use of available resources. Common scheduling techniques include Round Robin, First Come First Served, and Shortest Job First
- **Queuing Theory:** Queuing theory is a mathematical approach to analyzing the performance of queuing systems, which can be used to model RA in Cloud computing
- **ML or DL:** ML or DL techniques, such as clustering and prediction models, can be used to predict resource usage and optimize RA in real-time

2.2 VM Placement Formulation

VM placement algorithms primarily aim to reduce energy consumption. As explained in [13], the goals include:

- Maximising usage of available resources
- Saving power by shutting down unnecessary servers

VM placement is NP-hard [2]. It is typically solved using exact algorithms such as LP, ILP, MILP, and CP. Metaheuristics such as PSO, ACO, and GA are also used. Recent studies apply machine learning methods such as RL and DRL.

2.3 Mathematical Formulation

The mathematical formulation of the classical bin packing problem is as follows [14]. Let $\{1, 2, \dots, n\}$ be the set of n items that have to be packed in the bins, with a_i being the size occupied by item i . Let $S = \{S_1, S_2, \dots, S_m\}$ be the set of m bins, where $S_i \subseteq \{1, 2, \dots, n\}$ is the set of items packed in bin i , and each bin has the same size V . The goal is to find the integer number of bins $B \leq m$ and a B partition of the set $(1, 2, \dots, n)$ given by, $S_1 \cup S_2 \cup \dots \cup S_B$ so that, $\sum_{i \in S_k} a_i \leq V, \forall k = 1, 2, \dots, B$. The optimal solution is to find minimal B . The integer Linear Programming formulation of Bin Packing problem is shown in Equation 1,

$$\min B = \sum_{i=1}^n y_i \quad (1)$$

subject to

$$\begin{aligned} B &\geq 1 \\ \sum_{j=1}^n a_j x_{ij} &\leq V y_i, \forall i \in \{1, 2, \dots, n\} \\ \sum_{j=1}^n x_{ij} &= 1, \forall j \in \{1, 2, \dots, n\} \\ y_i &\in \{0, 1\}, \forall i \in \{1, 2, \dots, n\} \\ x_{ij} &\in \{0, 1\}, \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, n\} \end{aligned}$$

Where $y_i = 0$ if bin i is not used, otherwise, $y_i = 1$ and $x_{ij} = 0$ if item j is put into bin i , otherwise $x_{ij} = 1$

Virtual Machine Placement BP In the context of Virtual Machine Placement (VMP), we have a similar BP problem. The VMs are the objects of different sizes that are placed into hosts (bins) to ensure a minimum number of hosts are employed for placing all VMs.

3 Project Idea

The effectiveness of cloud computing relies on the importance of resource allocation [15, 3]. This encompasses many different challenges like load balancing, task scheduling, energy efficiency, autoscaling, and quality of service optimisation [15, 3, 2, 5].

Models based on machine learning (ML) have demonstrated significant promise in handling complex and diverse cloud environments, especially in the case of dynamic and/or real-time processing of data [15]. There is a growing need to not only react to dynamic conditions, but also anticipate future demands.

Idea: VM allocation focusing on IaaS models, as it often has a direct impact on performance, cost efficiency, and scalability. Would like to explore how RL agents can do dynamic resource allocation as it is still mostly heuristic.

Options (for the end goal, starting point will be much simpler):

- **Option A: Neural Combinatorial Optimisation (NCO) for Dynamic Vector Bin Packing.** Build a neural combinatorial optimiser that learns to place VMs on PMs under multiple resource constraints, formulated as a dynamic vector bin packing problem (DVBP). The goal is to demonstrate that a neural combinatorial optimiser can match or outperform DVBP heuristics and evolutionary methods - which are more commonly used. Moreover, we can show improved generalisability to unseen data.
- **Option B: Online resource allocation via RL/NCO with realistic dynamic workloads.** Build a reinforcement learning or NCO-based agent that performs online resource allocation (scaling or placement) under dynamic, uncertain workloads. The goal is to show that an RL/NCO agent can learn an online RA policy that outperforms heuristics and evolutionary baselines on dynamic workloads. Moreover, it can generalise better.

Hopefully, we can address one of the repeated issues:

- Heuristics and evolutionary algorithms dominate with static workloads but fail under dynamic workloads.
- VM placement (DVBP) remains heuristic-heavy and under-explored by ML. (harder)
- Generalisation to unseen workloads is a major open problem

ChatGPT: "Using the papers and research I have done [insert latex pdf], expand on what my two projects could be"

Option A: Neural Combinatorial This project formulates virtual machine (VM) placement as a Dynamic Vector Bin Packing (DVBP) problem, where each VM request arrives online with multi-dimensional resource demands (e.g., CPU, RAM, bandwidth) and must be assigned to a physical machine (PM) under capacity constraints. Existing DVBP heuristics (First-Fit, Best-Fit, Greedy, and DVBP variants) and evolutionary algorithms (GA, PSO, CSO) perform well in static or semi-static settings but struggle with real-world cloud data, workload variability, and handling multiple resources. **Most current approaches are heuristic and non-adaptive**

The project will develop a neural combinatorial optimisation (NCO) model—such as a pointer network, transformer-based policy, or graph neural network—that outputs placement decisions for incoming VMs requests. The model will be trained via reinforcement learning. Evaluation of the model will be on real-world data measuring things like SLA violation rate, energy efficiency, and/or load balancing.

Goal: Demonstrate that an NCO-based DVBP solver can match or outperform state-of-the-art heuristics and evolutionary algorithms on realistic VM placement workloads, while exhibiting improved adaptivity to workload changes.

Option B: Online Resource Allocation via RL/NCO with Realistic Dynamic Workloads. This project treats cloud resource allocation as an online sequential decision-making problem under uncertainty. Workloads fluctuate over time, and resource allocation must balance SLA satisfaction, energy efficiency, utilisation, and stability. Most existing approaches rely on threshold-based autoscaling or heuristic scheduling, which perform

poorly under bursty or unpredictable workloads. Evolutionary optimisation methods provide stronger baselines but remain slow, non-adaptive, and unsuitable for real-time decision-making.

The project will design a simulated cloud environment with dynamic workloads and multi-resource constraints. An RL agent (e.g., PPO, DQN, A2C) or NCO policy will learn to make online decisions such as scaling up/down, placing incoming tasks or VMs, and optionally migrating workloads. The placement subproblem may be handled via a DVBP heuristic or a simplified NCO module. Baselines will include threshold autoscaling, heuristic placement (FF/BF), and evolutionary RA methods. Evaluation metrics include SLA violation rate, response time or makespan, energy proxies (active PM count), and the stability of scaling/migration actions.

4 Implementation Research

[16] paper is about using an exact model based on linear integer programming to identify the most appropriate strategies and decisions for providers involved in cloud federations according to prices and user demands. Their system model involves encoding the demands of end-users into vectors

$$(d_u^{(small)}, d_u^{(large)}, d_u^{(xlarge)}, \dots),$$

Where the small, large and extra large instances reflect the common practice of current cloud service providers. However to simplify the model, they used small instances only. They also have a few mathematical formulations (1) to (11); however, their end goal does not seem as relevant, aiming more at maximising revenue for cloud providers based on if they insource, outsource, etc. However, if considering cost optimisation these equations could be useful.

[17] talks about using the meta-heuristic particle swarm optimization. They aimed to develop a static cost-minimization, deadline constrained heuristic for scheduling a scientific workload application in a cloud environment. Their approach involves (not solely) on VM performance variation. They used PSO to produce a schedule defining not only the task to resource mapping, but also the number and type of VMs that need to be leased including their times.

They formulate their problem with workflows. A workflow application $W = (T, E)$ is modeled as a directed acyclic graph (DAG) where $T = \{t_1, t_2, \dots, t_n\}$ is the set of tasks and E is the set of directed edges. An edge $e_{i,j}$ of the form (t_i, t_j) exists if there is a data dependency between t_i and t_j , with t_i being the parent task. They use this definition on the idea that a single task cannot be completed until its parent task is.

In reference to VMS, they assume that for every VM type, the processing capacity in terms of floating point operations per second (FLOPS) is available by the provider or can be estimated. They use this information to calculate the execution time of a task on a given VM. They model the performance variation by adjusting the processing capacity of each leased VM and introducing a performance degradation percentage $degv_M$. The execution time $ET_{t_i}^{VM_j}$ of a task t_i in a VM of type of type VM_j is estimated using the size I_{t_i} of the

task in terms of FLOP, depicted as

$$ET_{t_i}^{VM_j} = I_{t_i} / (P_{VM_j} \cdot (1 - deg_{vm_j}))$$

$TT_{e_{ij}}$ is defined as the time it takes to transfer data between a parent task t_i and its child t_j and is calculated as,

$$TT_{e_{ij}} = d_{t_i}^{out} / \beta$$

Where we assume that the size of the output data $d_{t_i}^{out}$ produced by task t_i is known in advance and that the entire workflow runs on a single data center or region. β is the bandwidth between each VM. Finally, the total processing time $PT_{t_i}^{VM_j}$ of a task in a VM is computed as,

$$PT_{t_i}^{VM_j} = ET_{t_i}^{VM_j} + \left(\sum_1^k TT_{e_{ij}} \cdot s_k \right)$$

Problem Definition (copy and pasted) for resource provisioning and scheduling Resource provisioning and scheduling heuristics may have different objectives; this work focuses on finding a schedule to execute a workflow on IaaS computing resources such that the total execution cost is minimized and the deadline is met. We define a schedule $S = (R, M, TEC, TET)$ in terms of a set of resources, a task-to-resource mapping, the total execution cost, and the total execution time. $R = \{r_1, r_2, \dots, r_n\}$ is the set of VMs that need to be leased; each resource r_i has a VM type VM_{r_i} associated to it as well as an estimated lease start time LST_{r_i} and lease end time LET_{r_i} . The mapping M is comprised of tuples of the form $m_{r_j}^{t_i} = (t_i, r_j, ST_{t_i}, ET_{t_i})$, one for each workflow task. A mapping tuple $m_{r_j}^{t_i}$ is interpreted as follows: task t_i is scheduled to run on resource r_j and is expected to start executing at time ST_{t_i} and complete by time ET_{t_i} . The following equations show how the total execution cost TEC and total execution time TET are calculated:

$$TEC = \sum_{i=1}^{|R|} C_{VM_{r_i}} \cdot \left\lfloor \frac{LET_{r_i} - LST_{r_i}}{\tau} \right\rfloor,$$

$$TET = \max\{ET_{t_i} : t_i \in T\}.$$

4.1 Implementing A Reinforcement Learning Agent

4.1.1 Introduction

RL is a decision-making agent that starts without any prior knowledge, learning through its own experience of repeated random interactions with an environment. Classically, an RL system involves an environment, agent, policy, reward, and value function [18, 19]. An agent is defined as a decision-making, goal-seeking virtual entity. An environment refers to everything that the agent can interact with, applying an action and receiving feedback. Policy π determines which action the agent will take when given a state from the environment. Reward is quantitative feedback given by the environment after the agent has done some specific action followed [19].

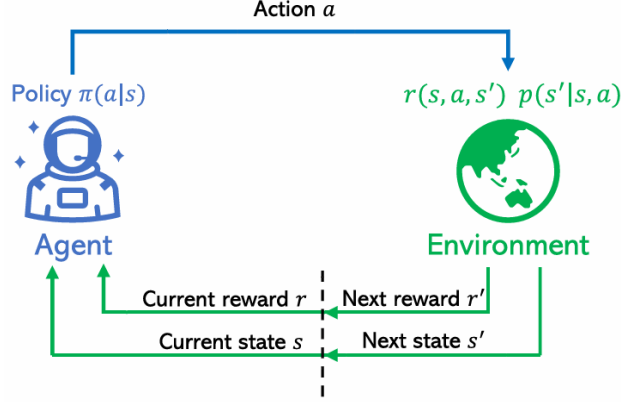


Figure 1: Agent Interaction With MDP

4.1.2 Markov Decision Process

The Markov Decision Process (MDP) is the foundational framework for decision in stochastic and uncertain environments [18]. It assumes the agent can observe a state s_i , and choose to take an action a_i . Then the agent moves to state s_{i+1} and continues. An MDP is described as a tuple of (S, A, P, R, γ) . S is a state space, the set of states. A is the action space, the set of actions the agent can select. P is a transition probability that satisfies the Markov property

$$p(s_{t+1}|s_1, a_1, s_2, a_2, \dots, s_t, a_t) = p(s_{t+1}|s_t, a_t)$$

Essentially, $p(s'|s, a) \in P$ describes the probability of transferring to a specific state s' by taking action a in the given state s . The reward function $R : S \times A \rightarrow \mathbb{R}$ gives the expected immediate reward $R(s, a) = \mathbb{E}[r_t | s_t = s, a_t = a]$ that the agent receives after taking action a in state s - alternatively this can be defined as $r(s, a, s') \in \mathbb{R}$ representing the reward received when transitioning from state s to s' via action a , where $R(s, a) = \sum_{s' \in S} p(s'|s, a)r(s, a, s')$. We have $\gamma \in (0, 1]$ as the discount factor that can be used to indicate short-term or long-term importance of rewards. The MDP loop can be seen in Figure 1. In principal, there exists a discrete time scale t where the next state s_{t+1} depends only on the state and action before it. The final goal of the agent is to maximise the long-term rewards.

The state-value function $V(s)$ can estimate the value of a state. It can be utilized to approximate how much long-term reward the agent can gain in all next states based on the current policy π by the agent,

$$V(s) = \sum_{a \in A} \pi(a|s) (R(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V(s'))$$

Given the agent follows policy π to take action a on state s to gain an immediate reward $R(s, a)$, over all possible states s' , we have a discounted sum of possible $V(s')$ (expected future returns) by adding a discount factor γ . Bellman optimality equations define the optimal state-value function $V^*(s)$ to get the best return for each state, as well as the optimal action-value function $Q^*(s, a)$ to get the maximum return for each state-action pair:

$$V^*(s) = \max_a (R(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V^*(s'))$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) \max_{a'} Q^*(s', a')$$

Types of methods

- Value-based: Q-learning is a typical off-policy learning that allows the agent to learn how to select actions in a given state to maximize total expected reward. The Q value (action-state pair) is updated by exploring possible actions of the state (via ϵ -greedy strategy) that maximizes said Q value. The update of the Q value is based on the current Q value, immediate reward, and maximum Q value of the next state. environment.[19, 18]
- Policy-based: Directly update the policy parameterized by θ on π with a gradient descent (Policy Gradient).
- Actor-Critic Network: A major DRL algorithm. The Actor learns the policy on how to take an action for a given state (the hyper-parameters set is θ). The critic evaluates the value of the policy by approaching an accurate value function using a neural network (hyper-parameter set ϕ).

4.1.3 Exploration in Reinforcement Learning

There are two main types of exploration. Efficiency where the explore is made more sample efficient so the agent can explore the environment in as few steps as possible. Safe exploration focuses on exploring the entire space. [19].

Potentially look at new reference [20] to add more detail.

5 Mathematical Formulation - Simpler Case

Consider the largest problem we would like to solve. There exists a cloud computing environment with a set of jobs (that may be dependent or independent) arriving on a time index at some unknown time that must be assigned to a set of physical machines. Each job has a processing duration and requires multiple resource types such as CPU, memory, GPU, bandwidth, or storage correlating to the dimensions considered in the Vector Bin Packing Problem. Jobs assigned to the same machine may overlap in time, provided that the total resource consumption of all simultaneously active jobs does not exceed the capacity of that machine. The total completion time of any given task is dependent on how much resources any physical machine provides it. For example, a machine of processing power 10 FLOPS. Say it holds 2 tasks requiring in total 20 FLOPS of processing each. The machine may allocate 8 FLOPS to one task and 2 FLOPS to another.

Now consider the simplest case. We have a set of independent jobs of which are all known and arrive at the same time, but whose starts can individually be delayed. They each have a set processing duration and deadline, and must be assigned to a set of identical physical machines. The time period will exist on integers only, i.e actions are done every second (not in between).

Table 1: Nomenclature

Sets	
J	Set of jobs, indexed by j
P	Set of processing jobs
M	Set of candidate machines, indexed by m
R	Set of resource types, indexed by r
T	Set of discrete time periods, indexed by t
Parameters	
p_j	Processing duration of job j
a_{jr}	Requirement of job j for resource type r
C_r	Capacity of each machine for resource type r
H	Length of the planning horizon
d_j	Due date or deadline of job j
w_j	Priority weight or penalty weight of job j
$\lambda_1, \lambda_2, \lambda_3$	Weights in the multi-objective function
Decision variables	
x_{jmt}	1 if job j starts on machine m at time t ; 0 otherwise
y_m	1 if machine m is used; 0 otherwise
s_j	Start time of job j
T_j	Tardiness of job j
U_{mrt}	Total usage of resource r on machine m at time t
θ	Maximum utilisation ratio over all machines, resources, and time periods

Let $T = \{0, 1, \dots, H\}$ be the planning horizon. Since job j must finish within the planning horizon, its feasible start times are $t = 0, \dots, H - p_j$, where p_j is the processing duration.

A good lower bound for H is:

$$H^{LB} = \max \left\{ \max_{j \in J} p_j, \left\lceil \frac{\sum_{j \in J} p_j}{|M|} \right\rceil, \max_{r \in R} \left\lceil \frac{\sum_{j \in J} a_{jr} p_j}{|M| C_r} \right\rceil \right\}.$$

A feasible upper bound of H is $H^{UB} = \sum_{j \in J} p_j$.

Assignment Constraint

Each job must be assigned to exactly one machine (no jumping around machines) and one start time (no stopping a job until it is completed):

$$\sum_{m \in M} \sum_{t=0}^{H-p_j} x_{jmt} = 1, \quad \forall j \in J. \quad (2)$$

Machine Constraint

A job can only be assigned to a machine if that machine is used (i.e we can not assign a job to machine that is off),

$$x_{jmt} \leq y_m, \quad \forall j \in J, m \in M, t = 0, \dots, H - p_j. \quad (3)$$

The start time of each job is defined as:

$$s_j = \sum_{m \in M} \sum_{t=0}^{H-p_j} t x_{jmt}, \quad \forall j \in J. \quad (4)$$

The resource load on machine m for resource type r at time t is given by:

$$U_{mrt} = \sum_{j \in J} \sum_{\tau=\max\{0, t-p_j+1\}}^{\min\{t, H-p_j\}} a_{jr} x_{jm\tau}, \quad \forall m \in M, r \in R, t \in T. \quad (5)$$

Capacity Constraint

The total usage of each resource must not exceed the capacity of the machine:

$$U_{mrt} \leq C_r y_m, \quad \forall m \in M, r \in R, t \in T. \quad (6)$$

The tardiness (how late a job finishes) of each job with respect to its deadline is defined as:

$$T_j \geq s_j + p_j - d_j, \quad \forall j \in J, \quad (7)$$

$$T_j \geq 0, \quad \forall j \in J. \quad (8)$$

The maximum utilisation ratio is defined as:

$$\theta \geq \frac{U_{mrt}}{C_r}, \quad \forall m \in M, r \in R, t \in T. \quad (9)$$

The binary and continuous variable domains are:

$$x_{jmt} \in \{0, 1\}, \quad \forall j \in J, m \in M, t = 0, \dots, H - p_j, \quad (10)$$

$$y_m \in \{0, 1\}, \quad \forall m \in M, \quad (11)$$

$$s_j \geq 0, \quad T_j \geq 0, \quad \forall j \in J, \quad (12)$$

$$U_{mrt} \geq 0, \quad \forall m \in M, r \in R, t \in T, \quad (13)$$

$$\theta \geq 0. \quad (14)$$

For a cloud computing application, we use the following weighted multi-objective function:

$$\min \lambda_1 \sum_{m \in M} y_m + \lambda_2 \sum_{j \in J} w_j T_j + \lambda_3 \theta. \quad (15)$$

The first term, $\sum_{m \in M} y_m$, minimises the number of active machines and therefore promotes server consolidation. The second term, $\sum_{j \in J} w_j T_j$, penalises weighted due date violations. The third term, θ , minimises the risk of hotspots/overload to spread the load more evenly over machines and time.

To reduce symmetry among identical machines, the following constraints may also be added:

$$y_m \geq y_{m+1}, \quad \forall m = 1, \dots, |M| - 1. \quad (16)$$

Instead of the weighted objective in (15), a lexicographic objective can also be used. In that case, the model first minimises the number of active machines, then minimises the total weighted violation, and finally minimises the maximum utilisation ratio:

$$\text{First:} \quad \min \sum_{m \in M} y_m, \quad (17)$$

$$\text{Second:} \quad \min \sum_{j \in J} w_j T_j, \quad (18)$$

$$\text{Third:} \quad \min \theta. \quad (19)$$

5.1 MDP formulation for the static offline bin packing case

In the complex case, this problem is modeled as resource-constrained scheduling instead of bin packing. This problem is considered a bin packing problem when each task is placed once and stays there forever (no time relation).

We now specialize the general MDP framework to the static, offline vector bin packing problem considered in the first stage of this project. In this simplest setting all items (VM requests) are known in advance, the agent will decide which tasks to assign to an existing physical machine sequentially. The problem can be expressed as an MDP (S, A, P, R, γ) where the components are defined as follows.

State space S . We work in time periods of integer lengths $t = 0, 1, 2, \dots$. At decision step t , a subset of jobs remains unscheduled, denoted $J_t \subseteq J$, and a subset of jobs that are currently being processed, $P_t \subseteq J$. A job will enter P_t when it gets assigned to a machine at decision step t , where it starts processing immediately and continues without interruption for p_j time. It leaves P_t once it completes (i.e $s_j + p_j = t$). Ideally, we have at the beginning,

$$J_0 = J \text{ and } P_0 = \emptyset$$

and at the end of the planning horizon all jobs completed,

$$J_{t_{final}} = \emptyset \text{ and } P_{t_{final}} = \emptyset$$

The number of jobs completed at any time t is,

$$|J| - |J_t| - |P_t| \tag{20}$$

For each unscheduled job $j \in J_t$, the state includes its features $F_j = (p_j, A_j, d_j, w_j)$. For each job $j \in P_t$, the state must include its assigned machine, start time, and its processing time $G_j = (m, s_j, p_j)$. In implementation however, all this will be encoded into one feature matrix F_t containing $(p_j, s_j, d_j, A_j, w_j, m_j)$ where the logical sets J_t and P_t can be derived from this representation.

Remaining machine capacity is time-dependent. For each machine $m \in M$ and resource $r \in R$, the state includes the remaining capacity vector at the current time index, denoted as R_{mrt} . The remaining capacity at time t is the total capacity of the machine minus the utilization from jobs in P_t that are active at time t ,

$$R_{mrt} = C_r - \sum_{j \in P_t: m_j=m} a_{jr} \tag{21}$$

We also include the binary machine-activation vector

$$y = (y_m)_{m \in M}, \tag{22}$$

where $y_m = 1$ if machine m has ever been assigned a job. This gives as a vector of all current active machines. In a more realistic scenario, machines can shut off when they have no remaining tasks. For simplicity, machines stay on once activated, which easily allows us to penalize the number of active machines. Thus the state at decision step t is

$$s_t = (J_t, P_t, \{F_j\}_{j \in J_t}, \{G_j\}_{j \in P_t}, \{R_{mrt}\}_{m \in M, r \in R}, y, t) \tag{23}$$

Action space $\mathcal{A}$. At each decision step t , the agent selects one unscheduled job and one machine to assign it to. The action set is

$$\alpha_t = \{(j, m) : j \in J_t, m \in M\}. \quad (24)$$

Choosing $\alpha_t = (j, m)$ means: schedule job j next on machine m , corresponding to $x_{jmt} = 1$. This moves job j from J_t to P_t , where it immediately starts processing and continues without interruption for p_j length of time. In doing so, we update $G_j = (m, s_j, p_j) \leftarrow (m, t, p_j)$, where, in implementation talk, m and s_j were previously some sentinel value (e.g. $+\infty$). The action (j, m) is feasible only if the resource-demand vector A_j fits within the remaining capacity of machine m :

$$a_{jr} \leq R_{mrt} \quad \forall r \in R. \quad (25)$$

If machine m is currently inactive (i.e. $y_m = 0$), selecting (j, m) implicitly opens/activates that machine permanently. If condition (25) is violated for any resource r , the action is considered infeasible. Infeasible actions remain in the action set α_t , but when selected will incur a large negative reward and the environment will not update any job sets or capacities.

Transition dynamics P . In the static offline setting, transitions are deterministic. Given the current state

$$s_t = (J_t, P_t, \{F_j\}_{j \in J_t}, \{G_j\}_{j \in P_t}, \{R_{mrt}\}_{m \in M, r \in R}, y, t) \quad (23)$$

and an action $\alpha_t = (j, m)$ with $j \in J_t$ and $m \in M$ (place job j on machine m), we have:

- **Job set update.** The selected job j is removed from the unscheduled set and added to the processing set

$$J_t^{new} = J_t \setminus \{j\}, \quad P_t^{new} = P_t \cup \{j\} \quad (26)$$

The start time of job j is set to the current time index, $s_j = t$, and its assigned machine is recorded in $G_j = (m_j, s_j, p_j)$ with $m_j = m$.

- **Time Advancement and Completion.** The decision step then advances to $t + 1$. Any job $j \in P_t^{new}$ that has now completed by time $t + 1$, i.e satisfies,

$$s_k + p_k = t + 1 \quad (27)$$

is removed from the processing set. Thus,

$$P_{t+1} = \{k \in P_t^{new} : s_k + p_k > t + 1\} \quad (28)$$

And the unscheduled set at time $t + 1$ is $J_{t+1} = J_t^{new}$

- **Machine capacity update.** The remaining capacity vector of each machine $m \in M$ for each resource $r \in R$ at time $t + 1$ is recomputed from the updated processing set P_{t+1} as

$$R_{mr,t+1} = C_r - \sum_{k \in P_{t+1} : m_k = m} a_{kr} \quad (29)$$

- **Machine activation update.** If machine m was previously inactive, it becomes active:

$$y_m = 1. \quad (30)$$

The activation status of all other machines is unchanged. In this way, y will record all machines that have been assigned at least one job up to time $t + 1$.

It is important to acknowledge that if action α_t is indeed an invalid placement breaking constraint (25), then only the time advancement and completion transition is completed, no machine activation, capacity or job set update takes place. A penalty gets added to the reward function.

Thus the next state is given by

$$s_{t+1} = (J_{t+1}, P_{t+1}, \{F_j\}_{j \in J_{t+1}}, \{G\}_{j \in P_{t+1}}, \{R_{mr,t+1}\}_{m \in M, r \in R}, y, t + 1),$$

and the transition kernel P is deterministic:

$$s_{t+1} = f(s_t, \alpha_t).$$

Reward function R . We want to align the learning with the multi-objective formulation in (15). For each decision step t , when the agent takes action $\alpha_t = (j, m)$ and assigns job j to machine m , we define an immediate reward, **Machine Activation Change**

$$r_t = -\lambda_1 \Delta y_t, \quad (31)$$

where

$$\Delta y_t = \sum_{m \in M} (y_m^{t+1} - y_m^t) \in \{0, 1\} \quad (32)$$

This term will penalize the activation of new machines, and therefore encourages consolidation.

Jobs Completing at $t + 1$. Given some set of jobs completing at time $t + 1$, say Γ . For each such job, its tardiness is,

$$T_j = \max\{0, (t + 1) - d_j\} \quad (33)$$

And its contribution to the objective is then $w_j T_j$

Hotspot / Utilisation Term. Let θ_t be the maximum utilisation ratio (worst ratio observed up to time t) induced by the schedule over all machines, and resources up to time t . In this sense, the maximum utilisation is simply $\theta = \theta_{t_{final}}$. We will define the increase,

$$\Delta \theta_t = \max\{0, \theta_{t+1} - \theta_t\} \quad (34)$$

so that we only get penalised for creating a new, worse hotspot.

Infeasible Action We define the binary variable $I(\alpha_t)$ that is 1 for an infeasible action and 0 otherwise. Given an invalid action, we provide an immediate penalisation $\lambda_{invalid}$.

Reward Function We will define the immediate reward at step t as,

$$r_t = -\lambda_1 \Delta y - \lambda_2 \sum_{j \in \Gamma_{t+1}} w_j T_j - \lambda_3 \Delta \theta_t - \lambda_4 I(\alpha_t) \quad (35)$$

This aligns with the objective function (15).

We then also consider a terminal penalty at the planning horizon for when $t = H$ (end of the episode). In such a case, we penalise all unfinished jobs,

$$r_H = -|J_t| \quad (36)$$

So that over one entire episode, the total return is,

$$r_H + \sum_{t=0}^{H-1} r_t \quad (37)$$

Discount factor γ . We will use an undiscounted return with $\gamma = 1$. The multi objective function (15) already sums the costs over the entire planning horizon with no timing preference. Thus, we have no need to encourage early or late improvements.

Episode termination. An episode terminates either when all jobs have completed, or when the planning horizon H is reached. If the latter is true with unfinished jobs, we apply a terminal penalty for each of the remaining jobs.

5.2 Simplest Case

A simple case is considered first: static, all items are known in advance, all items (jobs) are provided at start (i.e no jobs arrive during the entire episode).

5.2.1 Pseudocode of The Simplest Case

We have the setup of the deterministic, static offline case.

- J set of all jobs (deterministic and static)
- M set of all machine candidates
- R set of resource types
- T discrete time of the episode $T = \{0, \dots, H\}$
For each job,
 - p_j processing time

- a_{jr} resource requirements
- d_j deadline
- w_j weights

With parameters:

- C_r capacity of each machine for resource r
- H planning horizon

We form the state of the system into a matrix of row indexes $j \in J$ with rows,

$$(p_j, s_j, d_j, A_j, w_j, m_j) \quad (38)$$

In such a way the total computational size of jobs is proportional to $6 |A_{\forall j \in J}| |J|$. With this state formulation, all jobs are initialised to be unscheduled with some sentinel value like $+\infty$ as its start time. In that way, we deduce,

$$\begin{cases} j \in J_t & \text{if } s_t + p > t \\ j \in P_t & \text{if } s_t + p < t \end{cases} \quad (39)$$

We then define two machine matrices, the first being the total capacity C of each machine which remains static. Each row of this matrix represents machine index m where each column is resource index r ,

$$C = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1R} \\ C_{21} & C_{22} & \cdots & C_{2R} \\ \vdots & \vdots & \ddots & \vdots \\ C_{M1} & C_{M2} & \cdots & C_{MR} \end{bmatrix}, \quad (40)$$

This matrix remains static. The second matrix is the remaining machine capacity R . Each row of this matrix is the index m , where each column represents the remaining resource of index r .

$$R = \begin{bmatrix} R_{11} & R_{12} & \cdots & R_{1R} \\ R_{21} & R_{22} & \cdots & R_{2R} \\ \vdots & \vdots & \ddots & \vdots \\ R_{M1} & R_{M2} & \cdots & R_{MR} \end{bmatrix} \quad (41)$$

We allow the agent to take an action defined in the action space, $a_t = (j, m)$. If $y_m = 0$, we require the activation of that machine. We move onto the state transition pseudocode.

Algorithm 1 Transition (Step) Function

```
1: function STEP(action)
2:    $j \leftarrow$  action selected job
3:    $m \leftarrow$  action selected machine
4:   reward  $\leftarrow$  0
5:   if  $y_m = 0$  then                                      $\triangleright$  Machine activation (state update only)
6:      $y_m \leftarrow 1$ 
7:   end if
8:   if not ISFEASIBLE( $j, m, t$ ) then                          $\triangleright$  Feasibility check at current time  $t$ 
9:     reward  $\leftarrow -\lambda_{\text{invalid}}$ 
10:    return ( $state, reward, done = \text{False}$ )
11:  end if
12:  for each resource  $r \in R$  do                                $\triangleright$  Apply resource consumption
13:     $R_{mrt} \leftarrow R_{mrt} - a_{jr}$ 
14:  end for
15:   $s_j \leftarrow t$                                             $\triangleright$  Update job timing
16:   $T_j \leftarrow \max(0, s_j + p_j - d_j)$ 
17:   $J_{\text{rem}} \leftarrow J_{\text{rem}} \setminus \{j\}$                   $\triangleright$  Remove job from remaining set
18:  reward  $\leftarrow$  REWARD( $j, m, t, T_j$ )                    $\triangleright$  Compute reward for valid placement
19:   $t \leftarrow t + 1$                                           $\triangleright$  Advance time index
20:  if  $J_{\text{rem}} = \emptyset$  then                                $\triangleright$  Check termination
21:    done  $\leftarrow$  True
22:  else
23:    done  $\leftarrow$  False
24:  end if
25:  return ( $new\_state, reward, done$ )
26: end function
```

Algorithm 2 Reward Function

```
1: function REWARD( $j, m, t, T_j$ )
2:   reward  $\leftarrow$  0
                                      $\triangleright$  Machine activation penalty
3:   if  $m$  was activated at time  $t$  then
4:     reward  $\leftarrow$  reward  $- \lambda_1$ 
5:   end if
                                      $\triangleright$  Tardiness penalty
6:   reward  $\leftarrow$  reward  $- \lambda_2 w_j T_j$ 
                                      $\triangleright$  Load balancing penalty (maximum utilisation ratio)
7:   reward  $\leftarrow$  reward  $- \lambda_4 \theta$ 
8:   return reward
9: end function
```

We could add a utilisation reward next time to encourage packing jobs tightly:

$$\Delta u \leftarrow \sum_{r \in R} \frac{a_{jr}}{C_r}$$

$$reward \leftarrow reward + \lambda_3 \Delta u$$

Finally, we have the episode loop:

Algorithm 3 Episode Loop for the Scheduling MDP

```
1: Initialise environment state
2:  $t \leftarrow 0$ 
3: done  $\leftarrow$  False
4: while not done do
                                      $\triangleright$  Agent selects a job-machine pair
5:   action  $\leftarrow$  POLICY( $state$ )
                                      $\triangleright$  Environment transition
6:   ( $state', reward, done$ )  $\leftarrow$  STEP( $action$ )
                                      $\triangleright$  Update agent (RL algorithm dependent)
7:   AGENTUPDATE( $state, action, reward, state'$ )
8:    $state \leftarrow state'$ 
9: end while
```

6 Policy Exploration - [21]

A policy refers to any function that returns a feasible action for a problem [21]. Typically, Markov Decision Process models are solved by solving the corresponding system of Bellman

equations, finding the optimal policy π^* :

$$V^{\pi^*}(s) = \max_{a \in \mathcal{A}} \left\{ R(s, a) + \sum_{\forall s' \in S'} \mathbb{P}(s'|s, a) V^{\pi^*}(s') \right\} \quad (42)$$

However, we can simply try and maximize a cumulative reward ultimately serves the same job. We have two main approaches of solving a MDP to optimality, policy iteration and value iteration. The former fixes a policy and iterates over computing its value, and updating the policy based on this new value until the policy remains stable. The latter relies on similar steps, but aims to directly maximize the value functions and only updates the policy afterwards. It is equivalent to say that the finding the optimal value function is the same as finding the optimal policy. Finding an exact solution with these methods is complex, and mainly policy and value approximation are used.

Irrelevant of what notation used before, for this section only, the below notation will be used (just to help with creating notes):

- s: state
- a: action
- π : policy
- ϕ : basis function (derives features from state)
- θ : feature weights
- t: time epoch
- R: reward function
- V: value function

Policy Approximation In this approximation, we directly modify the policy itself. Two main subclasses of this approach are Policy Function Approximation (PFA) and Cost Function Approximation (CFA). PFA is essentially a parameterized function of the policy. Plugging in the state returns a feasible action e.g a linear PFA might be,

$$\pi^{PFA}(s_t|\theta) = \theta_0 + \theta_1\phi_1(s_t) + \theta_2\phi_2(s_t) + \dots \quad (43)$$

Here the main challenge will be finding the appropriate basis functions $\phi(s)$ that accurately capture the decision-making process. This method is typically done with a good insight into the structure of the problem. The drawback of PFA is parameter tuning becomes harder. CFA also directly searches the policy space. However, instead of returning an action, it solves an optimisation problem in a constrained action space. The simplest form of CFA is simply a greedy heuristic. AN example CFA is,

$$\pi^{CFA}(s_t|\theta) = \arg_{a \in \mathcal{A}} \max_{(s_t, \theta)} \tilde{A}_t^\pi(s_t, a|\theta) \quad (44)$$

Value Approximation Explicitly considers the downstream impact of current decisions, taking into account the full decision horizon. Value approximation may be suitable when the policy structure is not eminent or we cannot properly oversee the downstream effects of current decisions.

Value Function Approximation (VFA) represents downstream values as a function. One of the problems with the Bellman equation is that after taking an action, random events may lead us to many new states $s' \in S'$. Thus it is sensible to consider the value of all reachable states s' for each action, and the probability of ending up there. VFAs circumvent this problem by using a deterministic approximation function $V_t(s_t, a_t)$ instead of a stochastic expectation term. The most simplest VFA is a lookup table where for each state-action pair, we store the average observed values. An example form of VFA may be,

$$\pi_t^{VFA}(s_t) = \arg_{a_t \in \mathcal{A}} \max (R(s_t, a_t) + \tilde{V}_t(s_t, a_t)) \quad (45)$$

Continuing from the lookup table example, we typically want to resort to features that capture the essence of the state, which we can either manually design or extract with e.g auto-encoders. Thus, we capture the state-action values in compact functions e.g a critic network, and tune the feature weights by observation.

Direct Lookahead Approximation (DLA) typically requires a good understanding of the problem structure, although to some extent neural networks can alleviate this problem. Instead of deriving an explicit function, a DLA samples downstream samples.

Hybrid There are hybrid strategies that combine multiple classes. For example, the actor-critic framework contains elements of both PFA (the actor) and VFA (the critic)

6.1 REINFORCE Algorithms [22]

REINFORCE algorithms were introduced by Williams (1992) for associative reinforcement learning tasks, where the learner must perform an input-output mapping and receive immediate or delayed reinforcement. In the original formulation, each weight in the network is updated at the end of a trial according to

$$\Delta w_{ij} = \alpha_{ij}(r - b_{ij})e_{ij},$$

where r is the reinforcement signal, b_{ij} is a reinforcement baseline, and e_{ij} is the eligibility of weight w_{ij} . The update increases weights that correlate positively with higher reward and decreases those that correlate negatively. Episodic REINFORCE algorithms extend this idea to delayed rewards, where reinforcement is only provided after an episode of k time steps.

Modern Policy-Gradient Formulation The REINFORCE update can be expressed more generally using the policy-gradient theorem. The theorem states that the gradient of the expected return with respect to policy parameters is

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi}(s, a)].$$

Since the true action-value function $Q^\pi(s, a)$ is typically unknown, REINFORCE replaces it with the sampled Monte-Carlo return

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k,$$

yielding the practical gradient estimator

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t | s_t) G_t].$$

Baseline for Variance Reduction Williams (1992) showed that subtracting a baseline $b(s_t)$ reduces variance without introducing bias:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t | s_t) (G_t - b(s_t)).$$

Choosing $b(s_t) = V^\pi(s_t)$ yields the advantage function, connecting REINFORCE directly to actor-critic methods.

6.2 Policy Gradient Methods for RL with Function Approximation- [23]

The standard approach of approximating a value function and determining a policy from it leads to methods (like Q-learning, SARSA) that can diverge or behave unpredictably, even if the best approximations are found at each step before changing the policy. While value function can work well in many applications, it has several limitations:

1. Oriented towards finding deterministic policies
2. An arbitrarily small change in the estimated value of an action can cause it to not be selected

There is a need for a more stable method for continuous, large, or high-dimensional problems [23]. The core idea is to directly parameterise the policy, optimise the expected return, and apply gradient descent.

Policy Gradient Theorem Consider a standard RL framework in which a learning agent interacts with a Markov decision process. With function approximation, there are two useful ways of formulating the agent's objective. The first is the average reward formulation, ranking policies according to their long-term expected reward per step, $\rho(\pi)$,

$$\rho(\pi) = \lim_{n \rightarrow \infty} \frac{1}{n} E\{r_1 + r_2 + \dots + r_n | \pi\} = \sum_s d^\pi(s) \sum_a \pi(s, a) \mathcal{R}_s^a \quad (46)$$

where the state, action and reward are given by, respectively, s, a, r , and $d^\pi(s)$ is the stationary distribution of states under π . The value of a state-action pair given a policy is defined as,

$$Q^\pi(s, a) = \sum_{t=1}^{\infty} E\{r_t - \rho(\pi) | s_0 = s, a_0 = a, \pi\} \quad (47)$$

The second formulation has a designated start state s_0 , and we care only about the long-term reward obtained from it,

$$\rho(\pi) = E\left\{\sum_{t=1}^{\infty} \gamma^{t-1} |s_0, \pi\right\} Q^\pi(s, a) = E\left\{\sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k} |s_t = s, a_t = a, \pi\right\} \quad (48)$$

for γ discount factor. The policy gradient theorem holds for both formulations.

6.2.1 Policy Gradient Theorem

For any MDP, in either of the discussed formulations,

$$\frac{\partial \rho}{\partial \theta} = \sum_s d^\pi(s) \sum_a \frac{\partial \pi(s, a)}{\partial \theta} Q^\pi(s, a) \quad (49)$$

This equation gets modernized into,

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla \theta \log \pi_\theta(a|s) Q^\pi(s, a)] \quad (50)$$

by replacing Q^π , with the advantaged $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ for baseline V , we lower variance and bias. This paper is foundational in the Actor-Critic connection, where the actor with policy parameters θ , has a function approximator critic that provides either Q^π or A^π for the gradient update. It is proven that there is local convergence for policy-gradient methods with arbitrary differentiable function approximators. Value-based methods lack such guarantees.

Compatible Function Approximation A key contribution of the paper is the compatible function approximation condition. If the critic uses a linear function approximator,

$$\hat{A}(s, a) = \phi(s, a)^\top w \quad (51)$$

and the features satisfy,

$$\phi(s, a) = \nabla_\theta \log \pi_\theta(a|s) \quad (52)$$

then the actor-critic gradient estimate is unbiased and guaranteed to converge to a local optimum. This compatibility ensures that the critic's approximation aligns exactly with the policy's gradient structure.

Natural Gradient Connection Under compatible function approximation, the resulting update direction corresponds to the natural policy gradient, which accounts for the geometry of the policy space. This connection found in the paper is foundational for natural actor-critic, TRPO, and PPO policy-gradient methods.

7 Policy Gradient Algorithms

7.1 Q-learning and Deep Q-learning

Q learning (year 1989) is an older method that aims to learn the optimal action-value function $Q(s, a)$ directly by iteratively updating estimates $Q(s, a)$ using the Bellman optimality equation. It is a temporal-difference (TD) based learning model. The core update is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right],$$

where α is the learning rate. Over time, repeated updates converge to the optimal action-value function under suitable conditions.

Deep Q-Learning (year 2013), in the form of a Deep-Q-network (DQN), extends Q-learning by replacing the tabular action function with a neural network $Q_\theta(s, a)$ that can approximate values in large or continuous state spaces. Because NNs introduce instability, DQN uses techniques to stabilise learning such as; experience replay and target networks. The former stores transitions in a replay buffer and samples minibatches which reduces correlation between consecutive updates. The target network is periodically updated with a copy of the main network. The main network is trained to minimize the squared error between $Q_\theta(s_t, a_t)$ and y_t .

7.2 Actor-Critic Methods - [24]

Actor-only methods work with a parameterized family of policies. Here the parameters are updated in the direction of improvement based on the gradient of the performance with respect to the actor's parameters. This estimation is made directly with simulation, however, is prone to variance and each new gradient is independent of the other, so there is no 'learning'.

Critic-only methods rely exclusively on value function approximation and aim at learning an approximate solution to the Bellman equation, hopefully returning a near-optimal policy.

Actor-critic methods aim at combining the strong points of both methods. The critic uses approximation and simulation to learn a value function, used to update the actor's policy parameters in a direction of performance improvement. We view these algorithms as stochastic gradient algorithms on the parameter space of the actor. The policy gradient depends only on the inner product between the q-function and certain actor-derived feature vectors (Konda & Tsitsiklis). Therefore, the critic does not need to approximate the full q-function; instead it must approximate its projection onto the subspace spanned by the actor's gradient features.

7.2.1 Implementation

The actor has parameters θ and a policy $\pi_\theta(u|x)$. It uses the gradient $\nabla J_\theta(\theta)$ which can be written as an expectation involving the q-function $q_\theta(x, u)$ (the expected excess cost incurred over a certain renewal period of the Markov chain), and the policy gradient features $\psi_\theta(x, u) = \nabla_\theta \log \pi_\theta(u|x)$. Thus the gradient is essentially,

$$\nabla_\theta J(\theta) = \mathbb{E}[q_\theta(x, u)\psi_\theta(x, u)]$$

The critic does not learn q_θ directly. Instead learning the projection of q_θ onto the span of some features $\phi_\theta(x, u)$ chosen s.t,

$$\psi_\theta(x, u) \in \text{span}\{\phi_\theta(x, u)\}$$

The critic (as used in paper) uses Temporal-Difference (TD) learning (updating estimates based on other estimates, updating the value function after every step). Paper uses TD(1) which is equivalent to Monte Carlo but implemented incrementally, giving the exact projection needed for the actor's gradient. More general there is TD(λ). Actor critic methods converge to a local minima. Example pseudocode of the algorithm similar to that used in the paper is below. **NOTE, THERE ARE A WIDE VARIETY OF DIFFERENT ACTOR CRITIC METHODS**

7.2.2 Summary - AI MADE FROM NOTES

- Actor updates policy parameters using the gradient $\nabla_\theta J(\theta) = \mathbb{E}[q_\theta(x, u)\psi_\theta(x, u)]$.
- Critic does not learn the full q_θ ; instead learns its projection onto chosen features.
- Projection condition ensures the critic provides exactly the information needed for the actor.
- TD(1) learning supplies incremental value updates equivalent to Monte Carlo returns.
- Actor performs stochastic gradient ascent using the critic's projected value estimates.
- Method combines policy optimisation (actor) with value approximation (critic).
- Converges to a local optimum of the performance objective.
- Forms the conceptual foundation for modern deep actor-critic algorithms.

7.3 (2016) Modern Deep Actor-Critic Method: A3C Asynchronous Advantage Actor-Critic - [25]

Instead of learning a projection of the action-value function onto the actor-derived features, A3C uses a value function critic $V_\phi(s)$ with parameters ϕ and computes the policy gradient using an advantage estimator based on n-step temporal-difference learning. The advantage estimate is given by,

$$A(s_t, a_t) = R_t^{(n)} + \gamma^n V_\phi(s_{t+n}) - V_\phi(s_t),$$

where $R_t^{(n)}$ is the n-step return. This provides a lower-variance policy gradient compared to REINFORCE.

A3C also introduces multiple independent actors that act in parallel, interacting with separate environment instances respectively. These actors accumulate the gradients over multiple different trajectories, each periodically applied to a centrally maintained copy of θ (set of parameters for both the actor and critic). By doing this asynchronously, each actor has a

potentially unique set of parameters at any point in time, decreasing correlation and stabilizing learning. Moreover, A3C introduces entropy regularization to encourage exploration, using neural networks for both the actor and critic. Conceptually distinct from classical actor-critic methods that use linear function approximation and TD(1) or TD(λ) critics that learn a projection, and often performs much better (especially on modern hardware that can handle parallel processing).

Summary

- Uses multiple parallel actors to generate decorrelated trajectories.
- Each actor maintains local parameters but periodically updates shared global parameters.
- Employs n -step returns to compute lower-variance advantage estimates.
- Critic uses $V_\phi(s)$ to bootstrap future value estimates.
- Asynchronous updates reduce correlation and improve stability.
- Entropy regularisation encourages exploration and prevents premature convergence.
- Neural networks used for both actor and critic enable scalable function approximation.
- Overall: stable, parallelised actor-critic method suited for modern RL environments.

7.3.1 Algorithm

The algorithm for A3C is below:

Algorithm 4 A3C

Require: $n \in \mathbb{N}, \alpha \in (0, 1], \gamma \in [0, 1], t_{\text{MAX}} \in \mathbb{N}, T_{\text{MAX}} \in \mathbb{N}$

```
1: Initialize  $\theta$  and  $\phi$  at random
2: for  $i = 1, \dots, n$  do ▷ in parallel
3:   while  $T \leq T_{\text{MAX}}$  do
4:      $d\theta \leftarrow 0, d\phi \leftarrow 0$  ▷ reset gradients
5:      $\theta^{(i)} \leftarrow \theta, \phi^{(i)} \leftarrow \phi$  ▷ synchronize parameters on actors
6:      $s_t \leftarrow p_0$  ▷ sample start state
7:      $t_{\text{start}} \leftarrow t$ 
8:     while  $s_t$  not terminal and  $t - t_{\text{start}} \leq t_{\text{MAX}}$  do
9:        $a_t \sim \pi_{\theta^{(i)}}(a_t \mid s_t)$  ▷ sample action
10:       $(s_{t+1}, r_{t+1}) \sim P(s_t, a_t)$  ▷ sample next state and reward
11:       $t \leftarrow t + 1, T \leftarrow T + 1$ 
12:    end while
13:    if  $s_t$  is terminal then
14:       $R \leftarrow 0$ 
15:    else
16:       $R \leftarrow V_{\phi^{(i)}}(s_t)$  ▷ bootstrap if necessary
17:    end if
18:    for  $j = t - 1, \dots, t_{\text{start}}$  do
19:       $R \leftarrow r_j + \gamma R$ 
20:       $A \leftarrow R - V_{\phi^{(i)}}(s_j)$ 
21:       $d\theta \leftarrow d\theta + \nabla_{\theta^{(i)}} \ln \pi_{\theta^{(i)}}(a_j \mid s_j) A$  ▷ accumulate gradients
22:       $d\phi \leftarrow d\phi + \nabla_{\phi^{(i)}} (R - V_{\phi^{(i)}}(s_j))^2$  ▷ accumulate gradients
23:    end for
24:    update  $\theta$  and  $\phi$  using  $d\theta$  and  $d\phi$  via gradient ascent / descent
25:  end while
26: end for
```

7.4 (2015) Trust Region Policy Optimization (TRPO) - [25]

Trust Region Policy Optimisation (TRPO) is a policy gradient method designed to ensure stable updates to the new policy by limiting how much it can deviate to the one before it. It imposes a trust region constraint on the Kullback-Leibler (KL) divergence between them (the relative entropy). In addition, TRPO uses an off-policy correction to account for the mismatch between the old policy used to generate trajectories and the new policy busy being optimised, ensuring monotonic and stable updates for both policies. However, calculating the KL penalty at every point in the state space is impractical, so a trust region constraint is provided instead.

TRPO maximizes a surrogate objective based on importance sampling:

$$L(\theta) = \mathbb{E}_{s, a \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(a \mid s)}{\pi_{\text{old}}(a \mid s)} A_{\pi_{\text{old}}}(s, a) \right]. \quad (53)$$

This objective encourages actions with positive advantage while down-weighting actions with negative advantage.

To prevent overly large updates, TRPO enforces a trust region constraint on the average KL divergence:

$$\mathbb{E}_{s \sim \pi_{\text{old}}} [D_{\text{KL}}(\pi_{\text{old}}(\cdot|s) \parallel \pi_{\theta}(\cdot|s))] \leq \delta, \quad (54)$$

where δ is a small constant controlling the update size. This constraint ensures that the new policy remains close to the old one, improving training stability.

7.4.1 Practical Approximation and Update Rule

Because the trust region constraint cannot be solved exactly, TRPO uses a local approximation of both the objective and the KL divergence. The surrogate objective is linearised, while the KL constraint is approximated quadratically using the Fisher information matrix H . This yields the constrained optimisation problem:

$$\max_{\theta} \quad g^{\top}(\theta - \theta_{\text{old}}) \quad (55)$$

$$\text{s.t.} \quad \frac{1}{2}(\theta - \theta_{\text{old}})^{\top} H(\theta - \theta_{\text{old}}) \leq \delta, \quad (56)$$

where g is the policy gradient. Solving this produces a natural gradient update:

$$\Delta\theta = \sqrt{\frac{2\delta}{g^{\top} H^{-1} g}} H^{-1} g. \quad (57)$$

In practice, $H^{-1}g$ is computed using conjugate gradient, and a backtracking line search ensures the KL constraint is satisfied.

7.4.2 Algorithm Summary - AI MADE FROM NOTES

1. Collect trajectories using the current policy.
2. Estimate advantages $A_{\pi_{\text{old}}}$.
3. Compute the policy gradient g .
4. Approximate the Fisher information matrix via KL divergence.
5. Use conjugate gradient to compute the natural gradient direction.
6. Perform line search to enforce the KL constraint.
7. Update policy parameters and fit the value function.

TRPO is commonly used with the estimator used by A3C or the one used by PPO.

7.4.3 Algorithm

Algorithm 5 TRPO

Require: $\delta \in \mathbb{R}$, number of iterations T , batch size U

- 1: Initialize policy parameters θ and value parameters ϕ
 - 2: $t \leftarrow 0$
 - 3: **while** $t \leq T$ **do**
 - 4: **for** $i = 1, \dots, U$ **do**
 - 5: Sample action $a \sim \pi_\theta(\cdot | s)$ ▷ sample action
 - 6: Sample next state and reward $(s, r) \sim P(s, a)$ ▷ sample next state and reward
 - 7: Store (s, a, r) in dataset $\mathcal{D}$
 - 8: $t \leftarrow t + 1$
 - 9: **end for**
 - 10: Compute returns R and advantages A
 - 11: Compute surrogate objective $L_{\text{sur}}(\theta)$
 - 12: Compute KL divergence constraint $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta)$ ▷ KL loss
 - 13: Compute policy gradient $g = \nabla_\theta L_{\text{sur}}(\theta)$ ▷ policy gradient
 - 14: Compute Fisher information matrix F ▷ natural gradient metric
 - 15: Compute search direction $d = F^{-1}g$ ▷ natural gradient direction
 - 16: Perform line search on θ along direction d ▷ trust-region update
 - 17: Update value parameters ϕ by minimizing $(R - V_\phi(s))^2$ ▷ value function update
 - 18: **end while**
-

7.5 Proximal Policy Optimization (PPO) - [25]

Given the complexity of TRPO, PPO is designed to enforce comparable constraints on the divergence between consecutive policies during the learning process while simplifying the algorithm to not require second-order methods. This is done by heuristically flattening the gradients outside of an approximate trust region around the old policy.

PPO uses the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)},$$

which measures how much the new policy deviates from the old one for the same sampled action. The clipped objective limits this ratio to a small interval around 1, ensuring small update sizes. This provides a first-order approximation to the trust region used in TRPO, but with significantly lower computational cost.

PPO typically performs several epochs of minibatch updates on the same batch of data, improving sample efficiency compared to A3C. Advantage estimates are often computed using Generalised Advantage Estimation (GAE), which reduces variance while maintaining sufficiently low bias. As with A3C, entropy regularisation is commonly added to encourage exploration and prevent premature convergence to deterministic policies.

7.5.1 Algorithm

Algorithm 6 PPO

Require: Clipping parameter ϵ , number of iterations T , batch size U

- 1: Initialize policy parameters θ and value parameters ϕ
- 2: $t \leftarrow 0$
- 3: **while** $t \leq T$ **do**
- 4: **for** $i = 1, \dots, U$ **do**
- 5: Sample action $a \sim \pi_\theta(\cdot | s)$ ▷ sample action
- 6: Sample next state and reward $(s, r) \sim P(s, a)$ ▷ sample next state and reward
- 7: Store (s, a, r) in dataset $\mathcal{D}$
- 8: $t \leftarrow t + 1$
- 9: **end for**
- 10: Compute returns R and advantages A
- 11: Compute probability ratios $r_t(\theta)$
- 12: Compute clipped objective

$$L_{\text{clip}}(\theta) = \min(r_t(\theta)A, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A)$$

- 13: Compute value loss $(R - V_\phi(s))^2$ ▷ value function loss
 - 14: Compute entropy bonus $H(\pi_\theta)$ ▷ exploration bonus
 - 15: Compute gradients $d_\theta = \nabla_\theta(L_{\text{clip}} + c_1 L_V + c_2 H)$ ▷ compute policy gradients
 - 16: $d_\phi = \nabla_\phi L_V$ ▷ compute value gradients
 - 17: Update θ and ϕ using gradient ascent/descent
 - 18: **end while**
-

7.6 (2020) V-MPO - [25]

For previous algorithms, we aim to maximize expected rewards by selecting actions. For V-MPO, RL problems are formulated as probabilistic inference problems of estimating posterior policies that are consistent with a desired outcome. This problem is then solved with Expectation Maximization (EM). V-MPO is an on-policy variant of the Maximum a-posteriori Policy Optimization (MPO) which is off-policy. The V refers to learning the value function V instead of the action-value function Q .

V-MPO constructs a constrained optimization problem where the policy is updated to maximize the likelihood of high-return actions under a KL-divergence constraint... (rest is pretty complex in the paper). In the end, V-MPO replaces Q -based weighting with V -based advantage estimation.

7.6.1 Algorithm

Algorithm 7 V-MPO

Require: $\eta \in \mathbb{R}, \nu \in \mathbb{R}, \epsilon_\eta \in \mathbb{R}, \epsilon_\nu \in \mathbb{R}, U \in \mathbb{N}, T \in \mathbb{N}$

```

1: Initialize policy parameters  $\theta$  and value parameters  $\phi$  randomly
2:  $t \leftarrow 0$ 
3: while  $t \leq T$  do
4:   for  $i = 1, \dots, U$  do
5:      $a \sim \pi_\theta(\cdot \mid s)$  ▷ sample action
6:      $\beta(a \mid s) \leftarrow \pi_\theta(a \mid s)$ 
7:      $(s, r) \sim P(s, a)$  ▷ sample next state and reward
8:      $t \leftarrow t + 1$ 
9:     Store  $(a, s, r, \beta(a \mid s))$  in dataset  $\mathcal{D}$ 
10:  end for
11:  for all epochs do
12:    Compute returns  $R$  and advantages  $A$ 
13:    Compute weighted dataset  $\tilde{\mathcal{D}}$ 
14:     $L_\nu \leftarrow \frac{1}{|\mathcal{D}|} \sum_{\mathcal{D}} \nu(\epsilon_\nu - \text{sg}(D_{\text{KL}}(\pi_{\text{old}} \parallel \pi_\theta))) + \text{sg}(\nu) D_{\text{KL}}(\pi_{\text{old}} \parallel \pi_\theta)$  ▷ KL loss
15:     $L_\pi \leftarrow -\frac{1}{|\tilde{\mathcal{D}}|} \sum_{\tilde{\mathcal{D}}} \ln \pi_\theta(a \mid s) \psi(s, a)$  ▷ policy loss
16:     $L_\eta \leftarrow \eta \epsilon_\eta + \eta \ln \left( \frac{1}{|\tilde{\mathcal{D}}|} \sum_{\tilde{\mathcal{D}}} \exp \left( \frac{A}{\eta} \right) \right)$  ▷ temperature loss
17:     $d_\theta \leftarrow \nabla_\theta (L_\pi + L_\nu)$  ▷ compute policy gradients
18:     $d_\eta \leftarrow \frac{\partial L_\eta}{\partial \eta}$  ▷ compute temperature gradient
19:     $d_\nu \leftarrow \frac{\partial L_\nu}{\partial \nu}$  ▷ compute KL multiplier gradient
20:     $d_\phi \leftarrow \frac{1}{|\mathcal{D}|} \sum_{\mathcal{D}} \nabla_\phi (R - V_\phi(s))^2$  ▷ compute value function gradient
21:    Update  $\theta, \eta, \nu$ , and  $\phi$  using  $d_\theta, d_\eta, d_\nu$ , and  $d_\phi$  via gradient ascent/descent
22:  end for
23: end while

```

7.7 AI Summary

Q-Learning

Strengths: Simple, off-policy, converges under mild conditions.

Weaknesses: Not scalable; discrete actions only.

Use Cases: Small tabular environments.

Requirements: The problem must have a discrete action space and a fully observable state representation.

Deep Q-Learning (DQN)

Strengths: Handles high-dimensional inputs; stable via replay and target networks.

Weaknesses: Overestimation bias; discrete actions only.

Use Cases: Tabular data; image-based RL.

Requirements: The problem must have a discrete action space and allow off-policy learning with replayed experience.

Actor–Critic

Strengths: Lower variance than REINFORCE; combines value + policy learning.

Weaknesses: Requires compatible features; local convergence only.

Use Cases: Continuous control; theoretical foundations.

Requirements: The problem must support learning a value baseline and provide sufficiently frequent rewards for TD updates.

A3C

Strengths: Parallel actors; decorrelated data; scalable.

Weaknesses: Asynchronous noise; less sample-efficient than PPO.

Use Cases: Large environments; multi-core hardware.

Requirements: The problem must allow many independent environment instances to run in parallel for asynchronous exploration.

TRPO

Strengths: Very stable; monotonic improvement guarantees.

Weaknesses: Computationally expensive; second-order optimisation.

Use Cases: Robotics; strict stability tasks.

Requirements: The problem must justify large on-policy batches and benefit from strict KL-constrained policy updates.

PPO

Strengths: Simple; stable; efficient; widely used.

Weaknesses: Clipping may limit improvement.

Use Cases: General RL; continuous control.

Requirements: The problem must support on-policy sampling and provide stable advantage estimates for clipped policy updates.

V-MPO Summary

Strengths: Provides stable EM-based policy updates with KL-constrained trust-region behaviour, offering robustness without second-order optimization.

Weaknesses: More complex than PPO due to EM formulation and KL constraints, less sample-efficient than off-policy MPO, and not widely adopted.

Use Cases: Continuous-control tasks where PPO is stable but underperforms and TRPO is too computationally expensive, especially when trust-region-like stability is desired.

Requirements: An environment that supports on-policy sampling, a learnable value function for advantage estimation, continuous actions that benefit from KL-stabilised updates, and sufficiently smooth rewards for EM optimization.

7.8 Summary of Different Actor-Critic Methods - [26]

AI summary of webpage

Actor–Critic (AC)

How it works: Uses a stochastic policy updated via $\nabla_{\theta} \log \pi(a|s) \delta$, where the critic provides a TD error baseline.

Strengths: Simple; foundational; reduces variance compared to REINFORCE.

Weaknesses: Still unstable; sensitive to function approximation.

Use Cases: Small or moderate environments; continuous or discrete actions.

Requirements: Problem must allow learning a value baseline; environment must provide frequent rewards; works best when state and action spaces are not extremely large.

Advantage Actor–Critic (A2C)

How it works: Multiple agents collect experience in parallel and update synchronously using advantage estimates.

Strengths: Lower variance; more stable than AC; parallel rollout improves learning.

Weaknesses: Requires parallel environments; higher compute.

Use Cases: Atari; environments where parallel simulation is feasible.

Requirements: You must be able to run many environment instances in parallel; reward signal must be dense enough for advantage estimation.

Asynchronous Advantage Actor–Critic (A3C)

How it works: Many agents run independently and asynchronously update a shared global model, decorrelating experience.

Strengths: Excellent exploration; fast training; decorrelated data.

Weaknesses: Asynchronous noise; less sample-efficient than PPO.

Use Cases: Large-scale tasks; multi-agent environments; CPU-heavy workloads.

Requirements: Must be able to run many asynchronous environments; problem should benefit from diverse exploration; works well when GPU is not required.

Deep Deterministic Policy Gradient (DDPG)

How it works: Deterministic actor $\mu(s)$ with a critic $Q(s, a)$ trained off-policy using replay buffers and target networks.

Strengths: Handles continuous actions; stable with proper tuning.

Weaknesses: Overestimation bias; brittle; sensitive to noise.

Use Cases: Robotics; continuous control tasks.

Requirements: Action space must be continuous; environment must tolerate deterministic exploration (Ornstein–Uhlenbeck noise); reward signal should be smooth.

Twin Delayed DDPG (TD3)

How it works: Uses twin critics to reduce overestimation, delayed actor updates, and target policy smoothing.

Strengths: Very stable; robust; fixes DDPG’s main issues.

Weaknesses: More compute due to twin critics.

Use Cases: High-stakes continuous control (drones, industrial control).

Requirements: Continuous action space; environment must benefit from stable Q-estimates; reward landscape should be smooth enough for deterministic policies.

Soft Actor–Critic (SAC)

How it works: Maximises reward + entropy; uses stochastic actors, twin critics, and automatic entropy tuning.

Strengths: Excellent exploration; robust; sample-efficient.

Weaknesses: More complex objective; entropy tuning adds overhead.

Use Cases: Complex continuous control; robotics; multi-task learning.

Requirements: Continuous action space; environment should benefit from high exploration; reward signal should not punish stochasticity.

Proximal Policy Optimization (PPO)

How it works: Uses a clipped probability ratio to prevent large policy updates; optimises a surrogate objective with minibatch SGD.

Strengths: Simple; stable; widely used; works in many settings.

Weaknesses: Clipping may slow improvement; sensitive to hyperparameters.

Use Cases: Games (Dota 2), physics simulations, general RL.

Requirements: Environment must allow on-policy sampling; reward signal should be stable enough for advantage estimation; works for both discrete and continuous actions.

Trust Region Policy Optimization (TRPO)

How it works: Constrains policy updates using a KL-divergence trust region; uses second-order optimisation.

Strengths: Very stable; monotonic improvement guarantees.

Weaknesses: Heavy computation; complex implementation.

Use Cases: Robotics; safety-critical continuous control.

Requirements: Environment must support large batch on-policy sampling; problem must justify expensive second-order optimisation; continuous actions preferred.

8 Determining the Best Policy To Implement

Selecting the best reinforcement learning algorithm depends on the properties of the environment and how much control we have of the problem. Each algorithm is used in its different cases, each making assumptions about the action space, sampling cost, reward structure, and stability requirements. We go ahead and systematically determine which algorithm is most appropriate to implement.

8.1 Action Space: Discrete vs Continuous

Certain algorithms require discrete or continuous actions. For our task scheduling problem, we defined the action space to be,

$$\alpha_t = \{(j, m) : j \in J_t, m \in M\}. \quad (24)$$

which is discrete. Hence when looking at the below table, we can only consider algorithms that work with our discrete set.

Table 2: Action-Space Compatibility of Algorithms Considered

Algorithm	Discrete Actions	Continuous Actions
Q-Learning	Yes	No
Deep Q-Learning (DQN)	Yes	No
Actor-Critic (Generic)	Yes	Yes
A2C	Yes	Yes
A3C	Yes	Yes
TRPO	Yes	Yes
PPO	Yes	Yes
V-MPO	Yes	Yes
DDPG	No	Yes
TD3	No	Yes
SAC	No	Yes

8.2 Problem Structure

In our setting, the agent selects an action in the discrete, finite action space. The state encodes important features like capacities, loads, ..., and the objective is to maximise long-term cumulative reward. The environment is stochastic (and non-stationary in the future case with varying job arrivals). With these characteristics we need algorithms that can:

- Work over discrete action spaces
- Remain stable under stochastic, and in the future non-stationary workloads
- Can optimise with long-horizon rewards

8.3 Value-based vs Policy-based

The value function based methods like Q-learning and DQN, while compatible with discrete actions, trying to optimise a value function as apposed to a policy in state spaces with complex dynamics, they often exhibit unstable training. Thus, we consider an actor-critic approach.

8.4 Comparing Actor-Critic Methods

We consider the actor-critic methods excluding DDPG, TD3 and SAC as they do not work on discrete action spaces. We have to examine their stability and implementation complexity with our problem definition.

A2C and A3C . Both A2C and A3C have its actors act in parallel. A2C waits for all actors before updating, and A3C has each actor update the global network independently [27]. A2C is more efficient on GPUs (batch processing), and A3C is efficient on multi-core CPUs (parallel execution). A2C is slightly more stable, reproducible, and simpler to implement (due to being deterministic unlike A3C and A3C suffering from noise)[26]. "While A3C was a groundbreaking algorithm ..., A2C (often implemented with GAE for advantage estimation) has become a very popular and strong baseline". A2C often achieves similar or even better results than A3C especially in GPU accelerated environments, and is much more easier to implement and debug.

TRPO . Trust Region Policy Optimisation enforces a hard KL-divergence constraint between successive policies, holding amazing stability. However, since it requires solving a second-order constrained optimisation problem for every update, it is computationally expensive and complex to implement.

V-MPO . V-MPO uses a more probabilistic view with expectation-maximisation updates and KL-weighted constraints (nts KL stands for Kullback-Leibler divergence, measuring how different two probability distributions are). It is stable and expressive but uncommon, and its implementation is more complex than first-order methods like PPO,.

PPO . Proximal Policy Optimisation replaces TRPO's hard trust region with a clipped surrogate objective. This clipping prevents destructive policy updates while retaining a simple first-order optimisation method. It is robust, easy to implement, and often performs well in most environments.

8.5 Final Algorithm Selection

Given the discrete action space, the stochastic and (soon to be) non-stationary workload, and the need for stable updates with long-horizon updates, two main algorithms will be implemented and compared. PPO as it offers,

- Strong stability with clipped updates

- Straightforward implementation against TRPO
- Proven performance across the RL research space

In addition to PPO, we will look at A2C as it provides,

- A simple synchronous actor-critic update
- Lower computational cost and simpler implementation than A3C or TRPO
- Proven performance

Other methods may still be used as baselines. By considering these two main algorithms, implementation remains feasible and results robust.

9 Implementation Status, Results Log, Code Changes...

This section will outline the current status and progress of the actual algorithms on the coding side. The AI summaries will be held on github here: [PUT LINK]
The current progress is as follows.

9.1 pre 24/07/26.

Implemented both PPO and A2C. Testing with the training and evaluation methods. Evaluation could not run kept encountering errors. These errors were due to the action masking only presenting infeasible actions i.e 0,0,0,0,0, and so the agent would stall. To fix this, an idling option has been given to the agent. The agent can decide to choose the very last job index $j_{final} + 1$ which will indicate it wants to idle. This will incur an idling penalty λ_{idle} and progress the time by a single time step. Future work and/or other iterations could involve allowing to schedule jobs in the future, tuning the penalty, allow to take stop a current job and place a new one ...

This idling mechanic provided a new symptom, idling was often the best action to take (was set to 0.5 and others penalties were at 1). Incurring a penalty only at the end of the entire horizon. This lead to the policy learning to place 100% probability on idling. Research by <https://arxiv.org/abs/2009.10897> identify failures of PPO. Clipping gradient updates in PPO can trap the policy into a suboptimal action early in training, preventing any large corrections. For larger discrete action spaces like our own, this problem "becomes increasingly problematic as the number of actions increases".

Some papers on flexible Job-shop scheduling <https://www.sciencedirect.com/science/article/abs/pii/S0957260125009070> decompose the scheduling decision into two smaller sub-decisions. Instead of flattening action (j, m) , it decomposes it to avoid the combinatorial blow-up it gives to the action space. Research on this is still ongoing.

Current fixes for the policies involve increasing the NN size of the policy networks. Potentially have an Optuna hyperparameter optimization to find the best hyperparameters for the policy networks.

9.2 Up to 14/08/26

Fixed the training.

The agent collapsed onto always picking the idling action. Reward reweighting for the idling just changed to which constant it collapsed to, not whether or not it collapsed - so there was a larger problem inside the code.

9.2.1 Lit Review

10 Notes to Self

Copied by: "[15]": Future research should explore hybrid architectures that integrate RL with DL, unsupervised learning, and evolutionary algorithms to enhance solution quality and computational efficiency [127]. Techniques such as transfer learning, meta-learning, and multi-agent RL will improve generalisation across varying problem scales and configurations. Additionally, adopting simulation-based training and interactive learning environments can accelerate model development and increase resilience in dynamic, uncertain conditions [63]. These improvements are essential for using theoretical ideas in practical, large-scale applications across different industries. Following these guidelines will make RL solutions for BPP more adaptable, sustainable, and efficient. This will encourage new ideas in research and practical applications.

After going through [15], the biggest relevant approaches already tackled, their results and limitations are:

Approach	Results	Limitations
AI/ML-based continuous monitoring	Real-time resource allocation responding to demand surges	Potential challenges in handling unpredictable workloads
RL-based task scheduling	Reduced response time and makespan, increased resource efficiency	May not handle real-time dynamic changes
ML-based resource allocation	Achieved efficient resource allocation with reduced execution time	Limited by chosen attribute values and bandwidth constraints
RL-based adaptive auto-scaling	Improved performance over default auto-scaling configurations	Limited to serverless computing; may not generalize well to other cloud models
RAPID framework for online policy learning	Achieved 9×improvements	Evaluation limited to specific workloads: requires broader applications for validation
Optimal RA using MIP	Reduced queuing time by 32% and increased training efficiency by up to 24%	
RL-based RA	Reduced mean CPU core allocation by 38.4-74.4%	Evaluation limited to specific benchmarks: real world applicability needs further validation
Meta-RL for predictive autoscaling	Achieved significant performance improvement in maintaining stable CPU utilization levels	Focused on Alipay's specific workloads; requires more research on generalization
RL-based adaptive auto-scaling	Improved CPU utilization and performance over default auto-scaling configurations	Limited to serverless computing; may not generalize well
Dynamic RA and memory management with ML	Improved resource utilization and reduced operational costs	Limited to simulated environments; real-world applicability needs validation
ML-based tool to predict job memory requirements	Improved accuracy in predicting memory requirements, aiding in efficient RA	Limited to high performance computing settings; applicability to general cloud environments not assessed.
ML models for workload forecasting and resource management	Enhanced prediction of workload patterns leading to better resource allocation	Survey-based study; lacks empirical validation in real-world scenarios
Dynamic resource allocation and memory management using deep convolutional NN	Achieved efficient resource utilization and reduced memory wastage	Limited to simulated setups; real-world performance yet to be tested
ML-based dynamic RA and memory management	Improved resource utilization and reduced operational costs	Limited to simulated environments; real-world applicability needs validation

The exact papers and their references can be seen in section 4.4 of [15].

11 Nomenclature

Abbreviation/Term	Description
BPP	Bin Packing Problem
DVBP	Dynamic Vector Bin Packing
IaaS	Infrastructure-as-a-Service
KPI	Key Performance Indicator
MDBP	Multidimensional Bin Packing
NCO	Neural Combinatorial Optimisation
PM	Physical machine
QoE	Quality of Experience
RA	Resource allocation
RL	Reinforcement learning
SLA	Service Level Agreement
VBP	Vector Bin Packing
VM	Virtual Machine
VMP	Virtual Machine Placement

References

- [1] H. Technologies, *Introduction to Cloud Computing Computing*. New York: Springer, 2022. [Online]. Available: <https://link.springer.com/book/10.1007/978-981-19-3026-3>
- [2] K. Saidi and D. Bardou, “Task scheduling and vm placement to resource allocation in cloud computing: Challenges and opportunities,” *Cluster Computing*, vol. 26, no. 5, pp. 3069–3087, 2023. [Online]. Available: www.proquest.com/docview/2918267010
- [3] A. Mohamed, M. Hamdan, S. Khan, A. Abdelaziz, S. F. Babiker, M. Imran, and M. N. Marsono, “Software-defined networks for resource allocation in cloud computing: A survey,” *Computer Networks*, vol. 195, p. 108151, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S1389128621002152>
- [4] N. Devi, S. Dalal, K. Solanki *et al.*, “A systematic literature review for load balancing and task scheduling techniques in cloud computing,” *Artificial Intelligence Review*, vol. 57, p. 276, 2024. [Online]. Available: <https://link.springer.com/article/10.1007/s10462-024-10925-w>
- [5] S. Alharthi, A. Alshamsi, A. Alseiari, and A. Alwarafy, “Auto-scaling techniques in cloud computing: Issues and research directions,” *Sensors*, vol. 24, no. 17, p. 5551, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/17/5551>
- [6] R. Makkar and N. Mangla, “Load balancing and task scheduling methods in cloud computing – a review,” in *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*. IEEE, 2024, pp. 1037–1043.
- [7] A. Khan, “Approximation algorithms for multidimensional bin packing,” Ph.D. dissertation, Georgia Institute of Technology, 2015. [Online]. Available: <https://www.proquest.com/dissertations-theses/approximation-algorithms-multidimensional-bin/docview/3275489033/se-2>
- [8] Z. Y. Lee and X. Tang, “Evaluation of dynamic vector bin packing for virtual machine placement,” in *arXiv preprint*, 2026. [Online]. Available: <https://arxiv.org/html/2602.14704v1>
- [9] M. S. Hasan and E. Huh, “Maximizing sla and qoe in heterogeneous cloud computing environment,” in *World Congress in Computer Science, Computer Engineering and Applied Computing (WorldComp)*. The Steering Committee of WorldComp, 2013. [Online]. Available: <https://www.proquest.com/conference-papers-proceedings/maximizing-sla-qoe-heterogeneous-cloud-computing/docview/1490902207/se-2>
- [10] M. H. Mohamaddiah, A. Abdullah, S. Subramaniam, and M. Hussin, “A survey on resource allocation and monitoring in cloud computing,” *International Journal of Machine Learning and Computing*, vol. 4, no. 1, pp. 31–38, 2014. [Online]. Available: <https://doi.org/10.7763/IJMLC.2014.V4.382>

- [11] V. Anuradha and D. Sumathi, "A survey on resource allocation strategies in cloud computing," *International Conference on Information Communication and Embedded Systems (ICICES2014)*, pp. 1–7, 2014.
- [12] Q. Zhang, L. Cheng, and R. Boutaba, "Cloud computing: State-of-the-art and research challenges," *Journal of Internet Services and Applications*, vol. 1, pp. 7–18, 2010. [Online]. Available: <https://link.springer.com/article/10.1007/s13174-010-0007-6>
- [13] M. H. Shirvani, A. M. Rahmani, and A. Sahafi, "A survey study on virtual machine migration and server consolidation techniques in dvfs-enabled cloud datacenter: Taxonomy and challenges," *Journal of King Saud University - Computer and Information Sciences*, vol. 32, no. 3, pp. 267–286, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1319157818302842>
- [14] S. Kumaraswamy and M. K. Nair, "Bin packing algorithms for virtual machine placement in cloud computing: A review," *International Journal of Electrical and Computer Engineering*, vol. 9, no. 1, pp. 512–524, 2019. [Online]. Available: <https://www.proquest.com/scholarly-journals/bin-packing-algorithms-virtual-machine-placement/docview/2205380596/se-2>
- [15] A. Rathee and S. Dalal, "A systematic literature review of machine learning-based resource allocation techniques in cloud computing," *Computing Archives for Informatics and Numerical Computation*, vol. 107, no. 9, p. 179, 09 2025. [Online]. Available: <https://www.proquest.com/scholarly-journals/systematic-literature-review-machine-learning/docview/3239203131/se-2>
- [16] M. Hadji and D. Zeghlache, "Mathematical programming approach for revenue maximization in cloud federations," *IEEE Transactions on Cloud Computing*, vol. 5, no. 1, pp. 99–111, 2017.
- [17] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioning and scheduling algorithm for scientific workflows on clouds," *IEEE Transactions on Cloud Computing*, vol. 2, no. 2, pp. 222–235, 2014.
- [18] W. Yang, A. Acuto, Y. Zhou, and D. Wojtczak, "A survey for deep reinforcement learning based network intrusion detection," *Applied AI Letters*, vol. 7, no. 2, p. 22, Jun 01 2026, copyright - © 2026. This work is published under <http://creativecommons.org/licenses/by/4.0/> (the "License"). Notwithstanding the ProQuest Terms and Conditions, you may use this content in accordance with the terms of the License; Last updated - 2026-03-17. [Online]. Available: <https://www.proquest.com/scholarly-journals/survey-deep-reinforcement-learning-based-network/docview/3314625396/se-2>
- [19] P. Ladosz, L. Weng, M. Kim, and H. Oh, "Exploration in deep reinforcement learning: A survey," *Information Fusion*, vol. 85, pp. 1–22, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253522000288>

- [20] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, “Deep reinforcement learning: A brief survey,” *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.
- [21] W. van Heeswijk, “The four policy classes of reinforcement learning,” *Towards Data Science*, May 2021, accessed: 2026-07-08. [Online]. Available: <https://towardsdatascience.com/the-four-policy-classes-of-reinforcement-learning-38185daa6c8a>
- [22] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, May 1992. [Online]. Available: <https://doi.org/10.1007/BF00992696>
- [23] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Advances in Neural Information Processing Systems 12 (NIPS 1999)*, S. A. Solla, T. K. Leen, and K.-R. Müller, Eds. MIT Press, 2000, pp. 1057–1063. [Online]. Available: https://papers.nips.cc/paper_files/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html
- [24] V. R. Konda and J. N. Tsitsiklis, “Actor-critic algorithms,” in *Advances in Neural Information Processing Systems*, vol. 12. MIT Press, 1999, pp. 1008–1014. [Online]. Available: <https://proceedings.neurips.cc/paper/1999/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf>
- [25] M. Lehmann, “The definitive guide to policy gradients in deep reinforcement learning: Theory, algorithms and implementations,” *arXiv preprint arXiv:2401.13662*, 2024, version 2, 1 Mar 2024. [Online]. Available: <https://arxiv.org/abs/2401.13662>
- [26] A. Greeks, “Types of actor-critic algorithms in reinforcement learning,” <https://aigreeks.com/types-of-actor-critic-algorithms-in-reinforcement/>, 2024, accessed: 2026-07-19.
- [27] ApX Machine Learning, “A2c and a3c algorithms,” <https://apxml.com/courses/advanced-reinforcement-learning/chapter-3-advanced-policy-gradients-actor-critic/a2c-a3c>, 2026, accessed: 2026-07-19.
- [28] V. V. Vinothina, R. Sridaran, and P. Ganapathi, “A survey on resource allocation strategies in cloud computing,” *International Journal of Advanced Computer Science and Applications*, 2012. [Online]. Available: <https://doi.org/10.14569/IJACSA.2012>
- [29] S. M. Parikh, “A survey on cloud computing resource allocation techniques,” in *2013 Nirma University International Conference on Engineering (NUiCONE)*. IEEE, 2013, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/NUiCONE.2013.6780076>
- [30] S. Bhatnagar and R. Mahant, “Dynamic resource allocation in cloud computing environments: Ai-driven approaches for optimizing workload distribution and cost efficiency,” in *Proceedings of International Conference on Paradigms of Communication, Computing and Data Analytics (PCCDA 2025)*, ser. Algorithms for Intelligent Systems, S. J. Nanda, H. Mittal, and M. H. Lim, Eds. Singapore: Springer, 2026. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-96-6843-4_23

- [31] A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X11000689?via%3Dihub>
- [32] G. Fox, S. Pallickara, S. K. Garg, S. Versteeg, and R. Buyya, “A framework for ranking of cloud computing services,” *Future Generation Computer Systems*, vol. 29, no. 4, pp. 1012–1023, 2013.
- [33] N. Dahmani, A. Nazir, I. Taleb, and S. M. S. Bukhari, “Reinforcement learning based intelligent optimisation for bin packing problems: A review,” *Array*, vol. 28, p. 100616, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590005625002437>
- [34] B. Wang, Z. Lin, W. Kong, and H. Dong, “Bin packing optimization via deep reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 2542–2549, 2025.
- [35] J. Zhang, B. Zi, and X. Ge, “Attend2pack: Bin packing through deep reinforcement learning with attention,” *CoRR*, vol. abs/2107.04333, 2021. [Online]. Available: <https://arxiv.org/abs/2107.04333>
- [36] M.-A. Chadia and H. Mousannifa, “Understanding reinforcement learning algorithms: The progress from basic q-learning to proximal policy optimization,” *arXiv preprint arXiv:2304.00026*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.00026>